

26-Introduction to NumPy: Concepts, Array Operations, and Practical Usage

Introduction to NumPy

- NumPy stands for Numerical Python and is a key package for numerical and scientific computing in Python.
- It is implemented in C for performance and provides an extensive library of array operations not available in Python's built-in array functionality.
- NumPy arrays and methods enable efficient storage and manipulation of large datasets, which is fundamental in data science.

Importance and Use Cases

- Virtually all data types (images, audio, video, text) can be represented as numerical arrays for manipulation.
- Example: An image can be loaded and converted into a NumPy array for operations like cropping or slicing; changes to the array directly affect the image.
- Many core Python data science libraries (pandas, scipy, scikit-image) are built on top of NumPy, utilizing its powerful array operations.
- NumPy provides a broader set of methods and better performance compared to Python's native array structures.

Array Creation and Data Types

Creating Arrays

- Arrays can be created from Python lists, tuples, or other iterables using `np.array()`.
 - The resulting type is `numpy.ndarray` (n-dimensional array), which differs from a standard list.
 - NumPy arrays require homogeneity—all elements must be the same data type.

Data Types and Upcasting

- NumPy auto-converts (upcasts) elements:
 - If elements are all integers, the array is integer.
 - If any element is float, the array becomes float.
 - Inclusion of a complex number upgrades all elements to complex type; inclusion of a string results in a string array; other objects yield an object type array.
- The `.dtype` attribute can be checked to confirm the data type.

Dimensions and Attributes

- `.size`: total number of elements.
- `.shape`: tuple representing dimensions (e.g., 1D: `(n,)`, 2D: `(rows, cols)`, 3D arrays).
- `.ndim`: number of array dimensions.

Array Creation Functions

- `np.arange(start, stop, step)`: Similar to Python's `range` but supports fractional steps.
- `np.linspace(start, stop, num_elements)`
: Generates evenly spaced numbers over an interval (step is calculated automatically).
 - Stop is inclusive; step size is computed as $(\text{stop} - \text{start}) / (\text{num_elements} - 1)$.
- `np.zeros(shape, dtype=float)`: Array of all zeros (default float, adjustable via `dtype`).
- `np.ones(shape, dtype=float)`: Array of all ones (similarly adjustable type).
- `np.eye(N, dtype=float)`: Creates an identity matrix (square matrix with ones on the diagonal).
- `np.empty(shape)`
: Creates an empty array (uninitialized values, memory garbage or reused data until overwritten).
- `np.random.randint(low, high, size)`
: Creates arrays filled with random integers in the given range.

Array Reshaping

- Arrays can be reshaped using `.reshape(new_shape)`
 - . The number of elements before and after must match.
 - If one dimension is set to a negative number (e.g., `-1`), NumPy infers its correct value automatically.
 - Only one dimension can be negative per reshape call.
 - Example: Reshape a (100,) array to (10, 10) or (4, 5, 5), as long as total elements remain unchanged.

- `.shape` can be set directly to change shape (e.g., `array1.shape = (25, 4)`).
- High-dimensional arrays (3D) support more complex storage (e.g., images with color channels).
- `.transpose()` or `.T` returns the transposed array (rows <-> columns).
- `.ravel()` flattens multi-dimensional arrays into 1D arrays.

Basic Operations on Arrays

- Arithmetic operations on arrays are performed element-wise:
 - Addition, subtraction, multiplication, division, exponentiation, modulo, etc.
 - Example: `array + 5` adds 5 to every element in the array.
- Comparisons and logical operations can be performed element-wise as well (e.g., `>`, `<`, `==`).
- Standard Python logical `and` / `or` do not work directly; instead, element-wise logical or bitwise operators must be used.
- Arrays must have matching shapes or the operation will fail (unless broadcasting rules allow compatibility).

Matrix and Vector Multiplication

- Element-wise multiplication (`*`) multiplies corresponding elements.
- Dot or vector product (`np.dot(array1, array2)`):

- For 1D arrays: computes the sum of element-wise products (classic dot product).
- For 2D arrays: standard matrix multiplication. The number of columns in the first array must equal the number of rows in the second.
- The resulting shape: (rows of first array, columns of second array).
- Calculation example provided in detail: sum across corresponding products, matching standard linear algebra rules.

Practice and Next Steps

- Lecturer emphasizes frequent practice of NumPy methods (creation, reshaping, operations) as these are foundational for working with pandas, matplotlib, seaborn, and for later topics (statistics and machine learning).
- Assignments and exercises will be provided for reinforcement.
- Strong understanding of these array fundamentals is crucial for efficient data science workflows.

Notes powered by [CraftNote](#)